

Spark: Cluster Computing with Working Sets

Matei Zaharia, Mosharaf Chowdhury, Michael J. Franklin, Scott Shenker, Ion Stoica
University of California, Berkeley

Abstract

MapReduce and its variants have been highly successful in implementing large-scale data-intensive applications on commodity clusters. However, most of these systems are built around an acyclic data flow model that is not suitable for other popular applications. This paper focuses on one such class of applications: those that reuse a working set of data across multiple parallel operations. This includes many iterative machine learning algorithms, as well as interactive data analysis tools. We propose a new framework called Spark that supports these applications while retaining the scalability and fault tolerance of MapReduce. To achieve these goals, Spark introduces an abstraction called resilient distributed datasets (RDDs). An RDD is a read-only collection of objects partitioned across a set of machines that can be rebuilt if a partition is lost. Spark can outperform Hadoop by 10x in iterative machine learning jobs, and can be used to interactively query a 39 GB dataset with sub-second response time.

1 Introduction

A new model of cluster computing has become widely popular, in which data-parallel computations are executed on clusters of unreliable machines by systems that automatically provide locality-aware scheduling, fault tolerance, and load balancing. MapReduce [11] pioneered this model, while systems like Dryad [17] and Map-Reduce-Merge [24] generalized the types of data flows supported. These systems achieve their scalability and fault tolerance by providing a programming model where the user creates acyclic data flow graphs to pass input data through a set of operators. This allows the underlying system to manage scheduling and to react to faults without user intervention.

While this data flow programming model is useful for a large class of applications, there are applications that cannot be expressed efficiently as acyclic data flows. In this paper, we focus on one such class of applications: those that reuse a *working set* of data across multiple parallel operations. This includes two use cases where we have seen Hadoop users report that MapReduce is deficient:

- **Iterative jobs:** Many common machine learning algorithms apply a function repeatedly to the same dataset to optimize a parameter (e.g., through gradient descent). While each iteration can be expressed as a

MapReduce/Dryad job, each job must reload the data from disk, incurring a significant performance penalty.

- **Interactive analytics:** Hadoop is often used to run ad-hoc exploratory queries on large datasets, through SQL interfaces such as Pig [21] and Hive [1]. Ideally, a user would be able to load a dataset of interest into memory across a number of machines and query it repeatedly. However, with Hadoop, each query incurs significant latency (tens of seconds) because it runs as a separate MapReduce job and reads data from disk.

This paper presents a new cluster computing framework called Spark, which supports applications with working sets while providing similar scalability and fault tolerance properties to MapReduce.

The main abstraction in Spark is that of a *resilient distributed dataset* (RDD), which represents a read-only collection of objects partitioned across a set of machines that can be rebuilt if a partition is lost. Users can explicitly cache an RDD in memory across machines and reuse it in multiple MapReduce-like *parallel operations*. RDDs achieve fault tolerance through a notion of *lineage*: if a partition of an RDD is lost, the RDD has enough information about how it was derived from other RDDs to be able to rebuild just that partition. Although RDDs are not a general shared memory abstraction, they represent a sweet-spot between expressivity on the one hand and scalability and reliability on the other hand, and we have found them well-suited for a variety of applications.

Spark is implemented in Scala [5], a statically typed high-level programming language for the Java VM, and exposes a functional programming interface similar to DryadLINQ [25]. In addition, Spark can be used interactively from a modified version of the Scala interpreter, which allows the user to define RDDs, functions, variables and classes and use them in parallel operations on a cluster. We believe that Spark is the first system to allow an efficient, general-purpose programming language to be used interactively to process large datasets on a cluster.

Although our implementation of Spark is still a prototype, early experience with the system is encouraging. We show that Spark can outperform Hadoop by 10x in iterative machine learning workloads and can be used interactively to scan a 39 GB dataset with sub-second latency.

This paper is organized as follows. Section 2 describes

Spark’s programming model and RDDs. Section 3 shows some example jobs. Section 4 describes our implementation, including our integration into Scala and its interpreter. Section 5 presents early results. We survey related work in Section 6 and end with a discussion in Section 7.

2 Programming Model

To use Spark, developers write a *driver program* that implements the high-level control flow of their application and launches various operations in parallel. Spark provides two main abstractions for parallel programming: *resilient distributed datasets* and *parallel operations* on these datasets (invoked by passing a function to apply on a dataset). In addition, Spark supports two restricted types of *shared variables* that can be used in functions running on the cluster, which we shall explain later.

2.1 Resilient Distributed Datasets (RDDs)

A resilient distributed dataset (RDD) is a read-only collection of objects partitioned across a set of machines that can be rebuilt if a partition is lost. The elements of an RDD need not exist in physical storage; instead, a handle to an RDD contains enough information to *compute* the RDD starting from data in reliable storage. This means that RDDs can always be reconstructed if nodes fail.

In Spark, each RDD is represented by a Scala object. Spark lets programmers construct RDDs in four ways:

- From a *file* in a shared file system, such as the Hadoop Distributed File System (HDFS).
- By “*parallelizing*” a Scala collection (e.g., an array) in the driver program, which means dividing it into a number of slices that will be sent to multiple nodes.
- By *transforming* an existing RDD. A dataset with elements of type A can be transformed into a dataset with elements of type B using an operation called *flatMap*, which passes each element through a user-provided function of type $A \Rightarrow List[B]$.¹ Other transformations can be expressed using *flatMap*, including *map* (pass elements through a function of type $A \Rightarrow B$) and *filter* (pick elements matching a predicate).
- By changing the *persistence* of an existing RDD. By default, RDDs are lazy and ephemeral. That is, partitions of a dataset are materialized on demand when they are used in a parallel operation (e.g., by passing a block of a file through a map function), and are discarded from memory after use.² However, a user can alter the persistence of an RDD through two actions:
 - The *cache* action leaves the dataset lazy, but hints that it should be kept in memory after the first time it is computed, because it will be reused.

¹*flatMap* has the same semantics as the *map* in MapReduce, but *map* is usually used to refer to a one-to-one function of type $A \Rightarrow B$ in Scala.

²This is how “distributed collections” function in DryadLINQ.

- The *save* action evaluates the dataset and writes it to a distributed filesystem such as HDFS. The saved version is used in future operations on it.

We note that our *cache* action is only a hint: if there is not enough memory in the cluster to cache all partitions of a dataset, Spark will recompute them when they are used. We chose this design so that Spark programs keep working (at reduced performance) if nodes fail or if a dataset is too big. This idea is loosely analogous to virtual memory.

We also plan to extend Spark to support other levels of persistence (e.g., in-memory replication across multiple nodes). Our goal is to let users trade off between the cost of storing an RDD, the speed of accessing it, the probability of losing part of it, and the cost of recomputing it.

2.2 Parallel Operations

Several parallel operations can be performed on RDDs:

- *reduce*: Combines dataset elements using an associative function to produce a result at the driver program.
- *collect*: Sends all elements of the dataset to the driver program. For example, an easy way to update an array in parallel is to parallelize, map and collect the array.
- *foreach*: Passes each element through a user provided function. This is only done for the side effects of the function (which might be to copy data to another system or to update a shared variable as explained below).

We note that Spark does not currently support a grouped reduce operation as in MapReduce; reduce results are only collected at one process (the driver).³ We plan to support grouped reductions in the future using a “shuffle” transformation on distributed datasets, as described in Section 7. However, even using a single reducer is enough to express a variety of useful algorithms. For example, a recent paper on MapReduce for machine learning on multicore systems [10] implemented ten learning algorithms without supporting parallel reduction.

2.3 Shared Variables

Programmers invoke operations like *map*, *filter* and *reduce* by passing closures (functions) to Spark. As is typical in functional programming, these closures can refer to variables in the scope where they are created. Normally, when Spark runs a closure on a worker node, these variables are copied to the worker. However, Spark also lets programmers create two restricted types of *shared variables* to support two simple but common usage patterns:

- *Broadcast variables*: If a large read-only piece of data (e.g., a lookup table) is used in multiple parallel operations, it is preferable to distribute it to the workers only once instead of packaging it with every closure. Spark lets the programmer create a “broadcast vari-

³Local reductions are first performed at each node, however.